1.问题概述

1.1 问题描述与分析

本次实验围绕大模型展开，需要在 Pytorch 框架下，基于已提供的 Stable Diffusion v1-5 预训练模型，在水墨画数据集上自行部署并训练 Stable Diffusion 大模型。

Stable Diffusion 是一个 2022 年发布的文本到图像的潜在扩散模型(Latent Diffusion Model），比最开始的 Diffusion Model 能够更快地根据文本提示 prompt 生成图像。主要由 3 个部分组成：**变分自编码器**（VAE）、**U-Net** 和**文本编码器**。整个模型的过程主要是**正向扩散过程**--为潜在数据（即全尺寸图像被编码成低维的数据，这也是其比直接处理图像的 Diffusion Model 更快的原因）添加噪声和**反向扩散过程**--去除潜在数据中的噪声。

本次实验中将主要对预训练模型的 **U-Net** 部分在提供的数据集上重新进行**全量微调**训练并更新其参数，VAE 和文本编码器将被冻结。

1.2 解决问题的思路和想法

根据以上的问题描述与分析，本次实验在 Stable Diffusion v1-5 预训练模型上进行微调（Fine-tuning），使其更好地适应由古诗文生成水墨画的任务。基于此，训练脚本的大概思路如下：

1. 数据处理阶段：本次实验使用提供的 HuggingFace 的 parquet 数据集。因此使用时需要通过 bytes 访问和 io 读取。提供的 parquet 数据集已经指定好了列--image 和 text，无需在训练脚本中设置。

2. 训练阶段：仅对 U-Net 部分进行训练，冻结 VAE 和文本编码器。并且合理设置学习率、批量大小等参数。

3. 训练结束后:将训练完的 U-Net 与原有的 VAE 和文本编码器重新组成完整的 Stable Diffusion Pipeline，在验证集和自定义 prompt 上进行古诗转水墨画的图像生成。

通过查阅一些资料，本次大模型的训练过程将采用 swanlab 进行训练过程的监控，训练过程中的 train-loss、leraning-rate、epoch、验证集图像生成、日志等记录均可通过访问 https://swanlab.cn/@wxj66666/SD-Paint4Poem/runs/xo2d3thnyu52f9jq2xy71/chart

来查看。

2.算法设计

本次整个训练并验证的过程主要由三个程序完成，sd_config.py 集中定义训练中需要的所有参数,train_sd1-5_Paint4Poem.py 是完整的大模型微调训练脚本,predit.py 则是加载训练好的模型，根据输入的 prompt 生成水墨画图片。sd_config.py 与 predit.py 的设计都非常简短且简单，在此不做过多赘述，主要对 train_sd1-5_Paint4Poem.py 整个训练脚本进行设计分析：

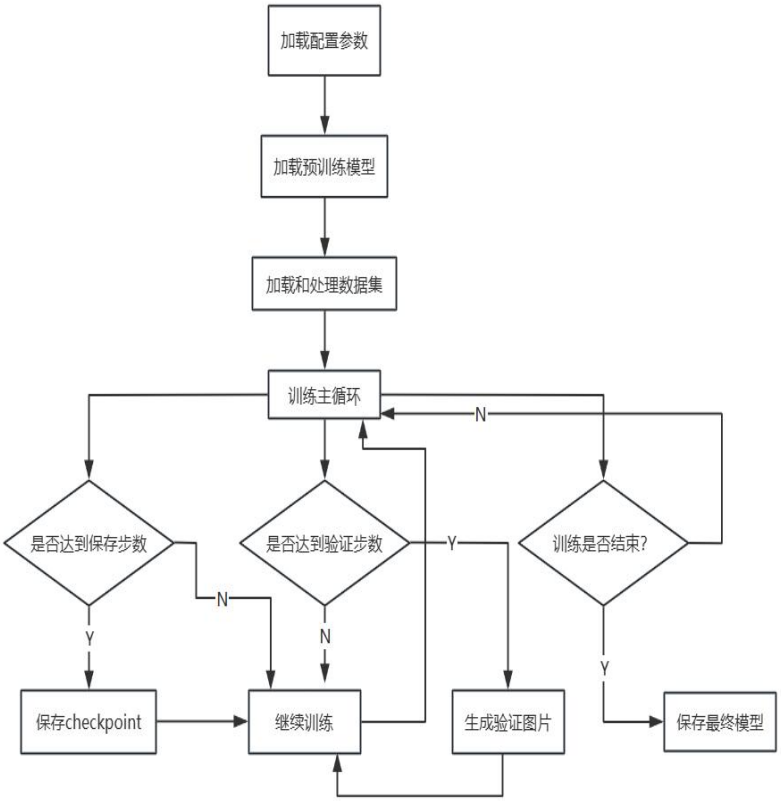
算法原理：

通过对 UNet 部分进行全量微调，在古诗水墨画的数据集上训练，使其能够根据输入的古诗文本生成对应风格的水墨画图像。训练过程中采用 VAE 对图片进行编码，Text Encoder 对文本进行编码，UNet 在 latent 空间（就是潜在数据组成的空间）预测噪声残差，优化 MSE 损失。

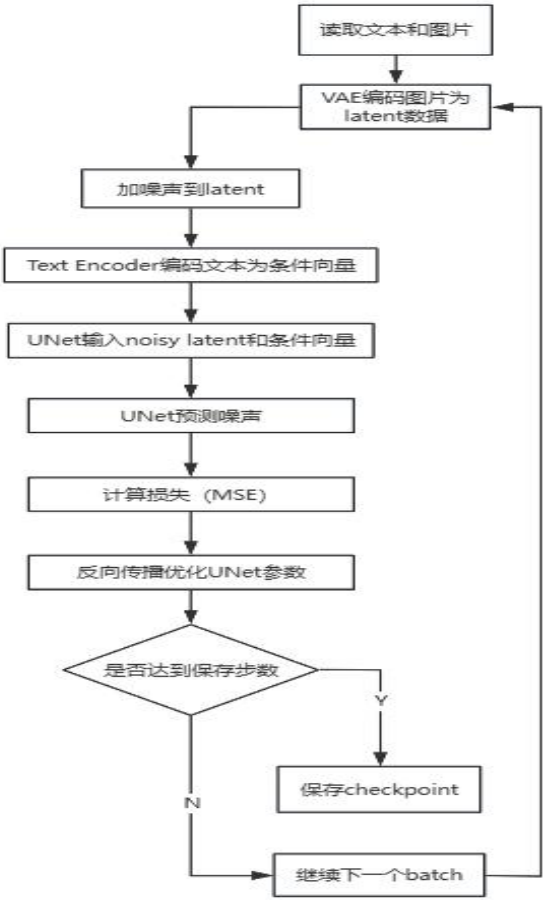
算法功能与设计思路：

1. **对古诗水墨画数据集进行处理：**将 parquet 格式的数据集解码为 PIL.Image，之后再经过 transforms 处理，变为模型输入的张量。
2. **冻结 VAE 和文本编码器，对 UNet 进行全量微调。**
3. **提供断点续训的功能：**定期保存 checkpoint，如果训练意外中断可以从保存的 checkpoint 继续训练（亲身实践，这个真的很有必要。具体可见“实验结果”中的描述）。
4. **通过 swanlab 对模型训练过程进行监控：**可以根据日志检查训练中可能出现的错误原因、查看 train-loss 的可视化图表和验证集图片、训练环境配置等等。
5. **训练完成后保存完整可用于推理的 pipeline 模型，便于后续的推理。**

算法流程图：



训练脚本的流程图



训练过程的流程图

3.算法实现

本次整个训练并验证的过程主要由三个程序完成，sd_config.py 集中定义训练中需要的所有参数,train_sd1-5_Paint4Poem.py 是完整的大模型微调训练脚本,predit.py 则是加载训练好的模型，根据输入的 prompt 生成水墨画图片。sd_config.py 与 predit.py 的设计都非常简短且简单，在此不做过多赘述，主要对 train_sd1-5_Paint4Poem.py 整个训练脚本进行设计分析：

- sd_config.py

实现细节：

在这个程序中定义了一系列训练需要的参数,使用 argparse 库定义和解析所有训练相关参数，使得相关参数既可以使用默认值，也可以从命令行指定。具体有：

(仅列出其中深度学习模型训练的常用参数，完整可见提交的源代码)

```
args.pretrained_model_name_or_path      # 预训练模型路径
args.dataset_name                       # 数据集路径
args.image_column                       # 数据集图片字段名
args.caption_column                     # 数据集文本字段名
args.validation_prompts                 # 验证用的 prompt 列表
args.output_dir                         # 输出目录(模型和 checkpoint 保存路径)
args.train_batch_size                   # 训练 batch size
args.num_train_epochs                   # 训练轮数
args.learning_rate                      # 学习率
args.adam_epsilon                       # Adam 优化器 epsilon
args.max_grad_norm                      # 最大梯度范数
args.prediction_type                    # 预测类型（epsilon/v_prediction）
args.checkpointing_steps                # checkpoint 保存步数
args.checkpoints_total_limit            # checkpoint 最多保留数量
args.resume_from_checkpoint             # 断点续训路径
args.validation_epochs                  # 验证间隔 epoch 数
```

核心函数及其输入输出：

1.parse_args()：定义/解析各个参数

```
def parse_args():
    parser = argparse.ArgumentParser(description="Simple example of a training script.")
    parser.add_argument(
        "--input_perturbation", type=float, default=0, help="The scale of input perturbation. Recommended 0.1."
    )
    #修改加载的模型和数据集路径#
    parser.add_argument(
        "--pretrained_model_name_or_path",
```

输入：无（修改时可在 sd_config.py 中直接修改或者命令行中输入）

输出：返回包含所有训练参数的 argparse.Namespace 对象

数据结构定义：

1.**参数对象（args）：**包含所有命令行参数

- predit.py

实现细节：

使用 HuggingFace 官方提供的 Diffusers 库中专门为 Stable Diffusion 设计的 pipeline 类--StableDiffusionPipeline 加载训练好的模型组件，然后输入想生成图片的 prompt，调用 pipeline，完成从文本生成图像的过程，并将生成的 png 图片保存在本地。

```
from diffusers import StableDiffusionPipeline
import torch

pipe = StableDiffusionPipeline.from_pretrained("./autodl-tmp/sd-model-finetuned", torch_dtype=torch.float16)
pipe = pipe.to("cuda")

prompt = "蕙风如薰，甘露如醴。"#修改成想生成的图片的promt
image = pipe(prompt, num_inference_steps=30).images[0]
image.save("蕙风如薰_甘露如醴-2.png")#可以自定义保存的图片名字
```

核心函数及其输入输出：

1. StableDiffusionPipeline.from_pretrained(str, torch_dtype)

输入：模型文件夹路径、torch_dtype

输出：返回 pipeline 对象

2. pipe(prompt, num_inference_steps=30)

输入：prompt（str），步数（int）

输出：返回生成图片（PIL.Image）

数据结构定义：

1. **str 字符串：**输入的 prompt
2. **PIL.Image 对象：** 进行各种图片操作

- train_sd1-5_Paint4Poem.py

实现细节：本训练脚本基于 HuggingFace diffusers、transformers、datasets、

accelerate 等库实现 Stable Diffusion 全量微调。其中 Diffusers 提供必要的模型组件和训练工具等，完成 SD 模型的训练与推理核心；transformers 则是提供必要的文本编码器和分词器，将验证集的 prompt 转为向量；datasets 负责数据集的加载和处理；accelerate 和 swanlab 平台集成，提供断点续训和训练记录等功能；torch/torchvision 则是提供基本的深度学习框架。

核心函数及其输入输出：

1. preprocess_train(examples)：

对样本图片进行解码和数据增强，对文本进行分词编码。

```
#处理dict数据集
def preprocess_train(examples):
    images = []
    for image in examples[image_column]:
        # 如果 image 是 dict 且有 'bytes' 字段，说明是 parquet 格式
        if isinstance(image, dict) and 'bytes' in image:
            img = Image.open(io.BytesIO(image['bytes'])).convert("RGB")
        else:
            # 否则直接认为是 PIL.Image
            img = image.convert("RGB")
        images.append(img)
    examples["pixel_values"] = [train_transforms(img) for img in images]
    examples["input_ids"] = tokenize_captions(examples)
    return examples
#-----------------
```

输入：examples（batch 样本 dict，含图片和文本字段）

输出：处理后的 dict，含 pixel_values 和 input_ids

2. tokenize_captions(examples, is_train=True)：

对文本批量分词并编码为定长序列

```
def tokenize_captions(examples, is_train=True):
    captions = []
    for caption in examples[caption_column]:
        if isinstance(caption, str):
            captions.append(caption)
        elif isinstance(caption, (list, np.ndarray)):
            # take a random caption if there are multiple
            captions.append(random.choice(caption) if is_train else caption[0])
        else:
            raise ValueError(
                f"Caption column `{caption_column}` should contain either strings or lists of strings."
            )
    inputs = tokenizer(
        captions, max_length=tokenizer.model_max_length, padding="max_length", truncation=True, return_tensors="pt"
    )
    return inputs.input_ids
```

输入：examples（dict）数据样本，is_train（bool）是否训练

输出：tokenizer 编码后的 input_ids

3. collate_fn(examples)：

将一个 batch 内的样本打包为张量格式的输入字典。

```python
def collate_fn(examples):
    pixel_values = torch.stack([example["pixel_values"] for example in examples])
    pixel_values = pixel_values.to(memory_format=torch.contiguous_format).float()
    input_ids = torch.stack([example["input_ids"] for example in examples])
    return {"pixel_values": pixel_values, "input_ids": input_ids}
```

输入：examples（dict）数据样本

输出：batch dict，含 pixel_values 和 input_ids

其他函数则均为 diffusers、transformers 等这些的库函数应用，来完成模型的训练。具体函数可见提供的训练源码。

数据结构定义：

1. **Dict 字典：**输入样本的图片
2. **Str 字符串：**输入样本的字符串文本
3. **Tensor 张量：**训练的 batch，均为 torch.Tensor

4.实验结果

本次实验需要显卡的显存 22GB 左右，但是个人电脑的 Nvidia 显卡的显存只有 16GB，没法在个人电脑上部署，因此选择在 AutoDL 平台上租了一张 rtx3090 的显卡（显存有 24GB），再通过 VSCode 远程连接部署和训练此次 SD 大模型。

基本配置为 PyTorch/2.3.0 /3.12(ubuntu22.04)/ 12.1。其余所需的库的版本详情可见 https://swanlab.cn/@wxj66666/SD-Paint4Poem/runs/xo2d3thnyu52f9jq2xy71/chart 中关于环境的记录。（这个链接打开后可以在 swanlab 上看到两次训练过程中的 train-loss 等所有记录图表）

```
Mon Jun  2 18:20:39 2025
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 525.125.06   Driver Version: 525.125.06   CUDA Version: 12.0     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla V100-SXM2...  Off  | 00000000:A3:00.0 Off |                    0 |
| N/A   38C    P0    41W / 300W |      0MiB / 16384MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
```

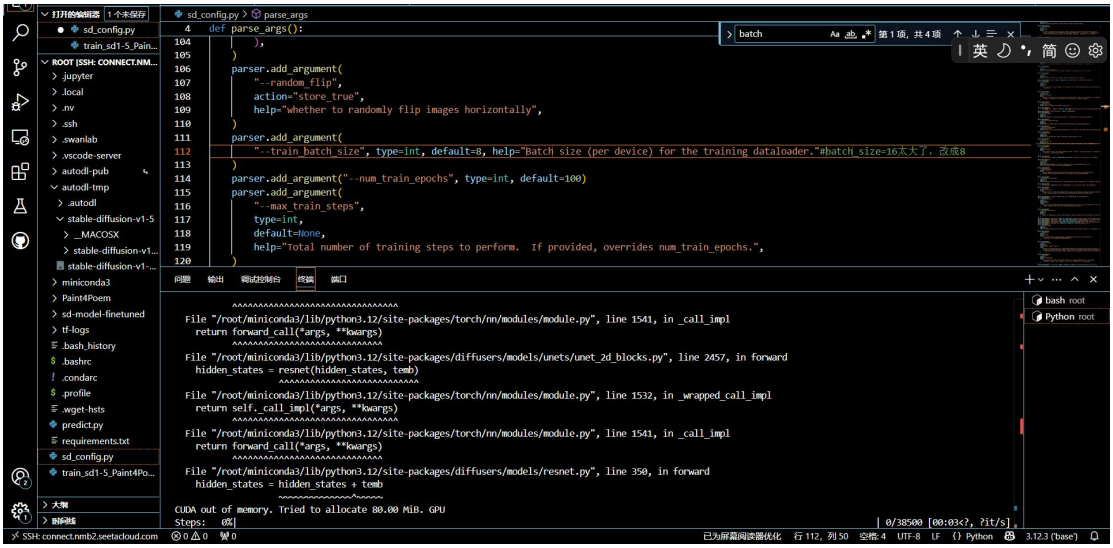
```
+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

PC 的显卡规格

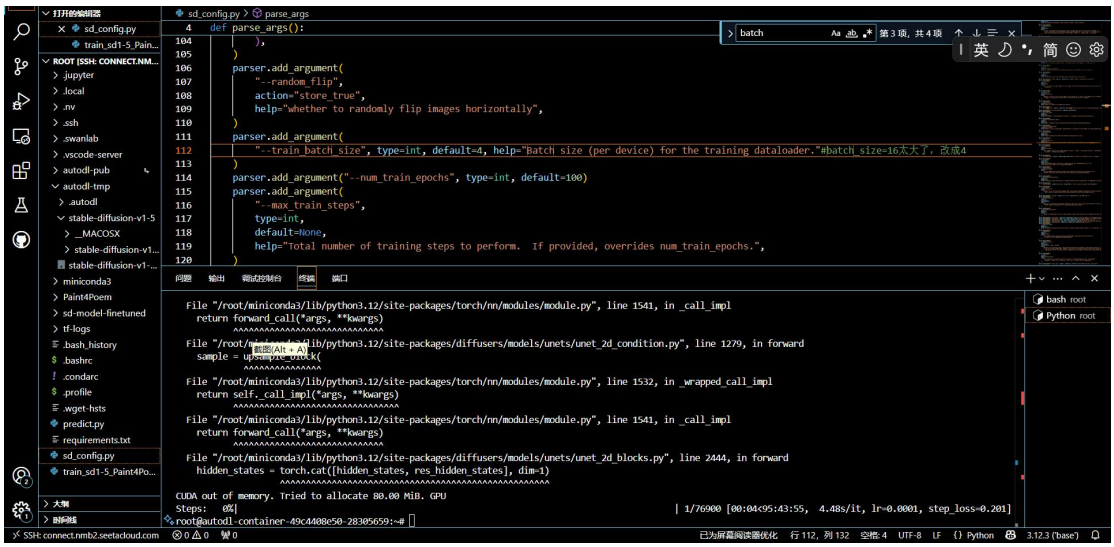
```
root@autodl-container-49c4408e50-28305659:~# nvidia-smi
Tue Jun  3 23:28:41 2025
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 570.124.04             Driver Version: 570.124.04     CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 3090        On  |   00000000:9D:00.0 Off |                  N/A |
| 30%   35C    P8             26W /  350W |       1MiB /  24576MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

rtx3090 的显卡规格

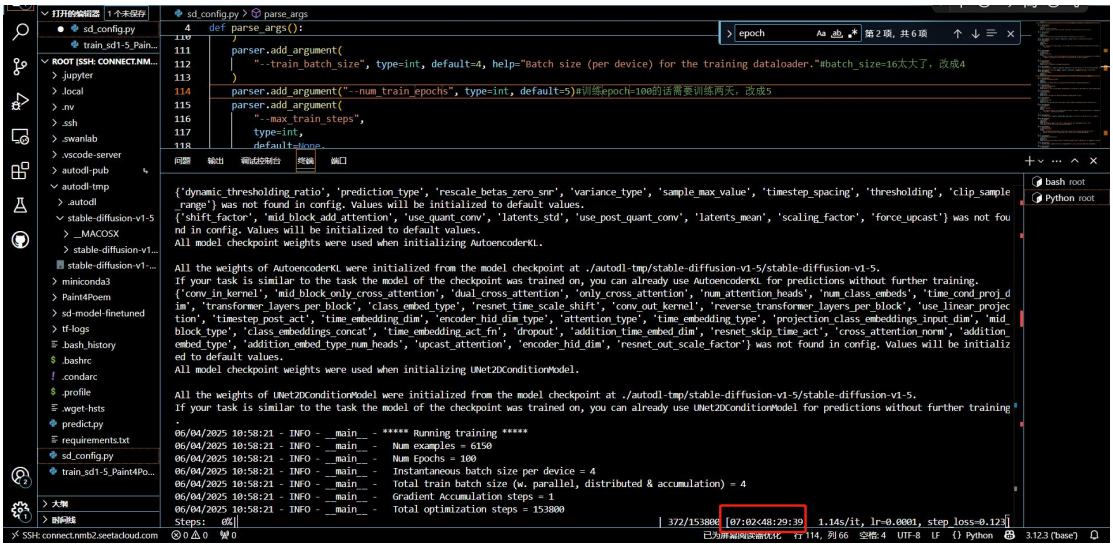
解决了环境配置和训练时出现的问题后（问题分析与解决见下一部分），开始进行模型的训练。第一次训练过程中 batch_size 设置为 4（其实试过 16、8，但是发现显存不够，没办法训练，4 是最大的 2 的 n 次方的 batch_size 了），epoch 设置为 5（epoch=100，大约需要 48h，epoch=5 则 2-3 个小时即可）。



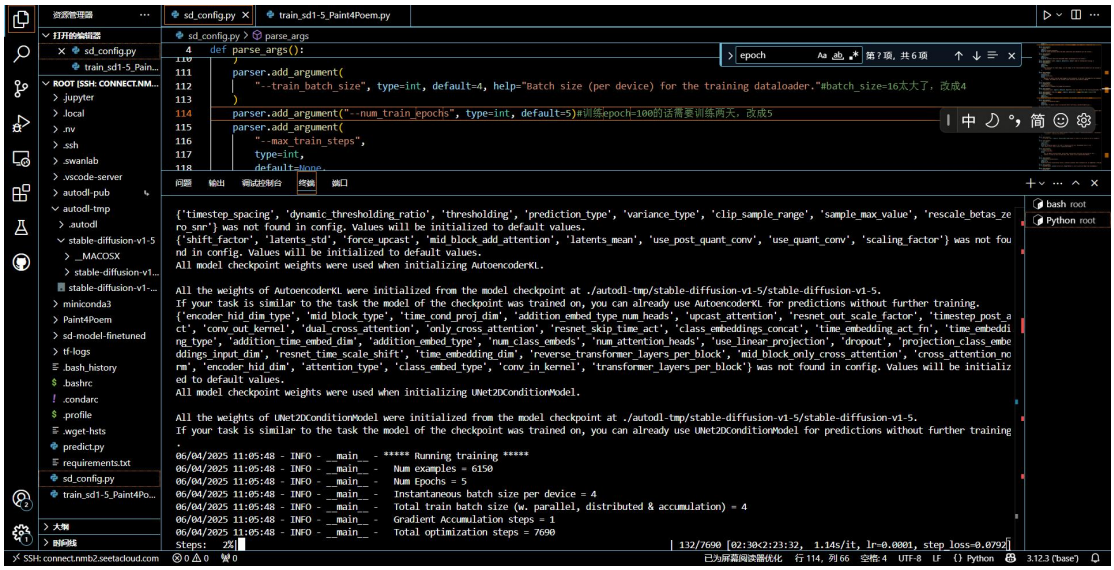
batch_size=16 时，显存不够报错



batch_size=8 时，显存不够依然报错

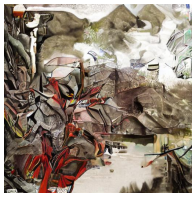
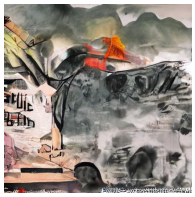


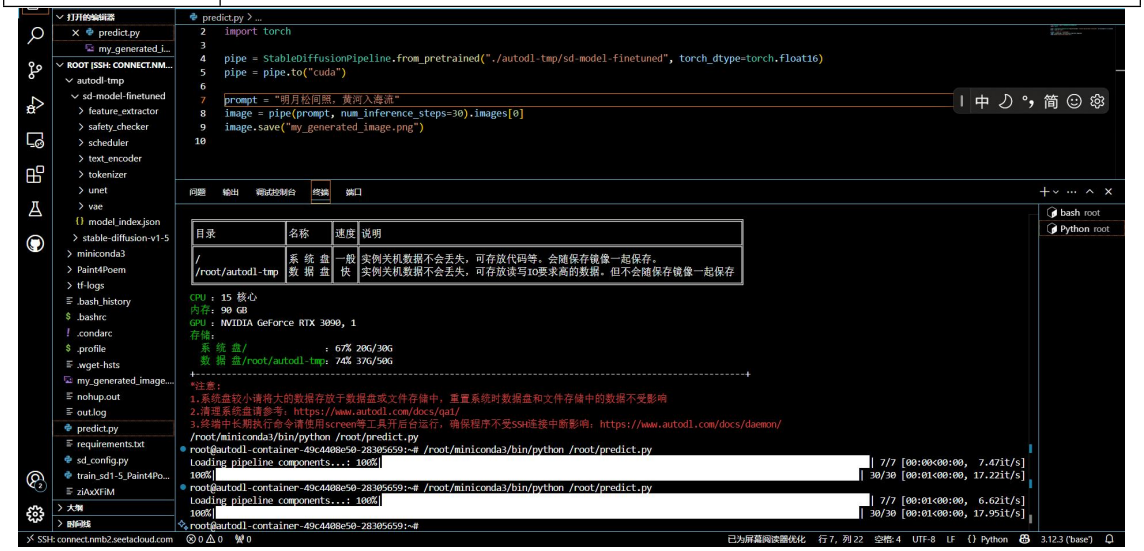
epoch=100 时，需要训练 48h



epoch=5 时，时间 2.5h 左右

以 epoch=5，batch_size=4 这组参数完成了第一次训练。但是第一次训练结果非常一般。生成的图像如下所示：（试了很多 prompt，画风都很诡异，像鬼片。。。）

图片	Prompt
	“孤帆远迎碧空尽，唯见长江天际流。”
	“蕙风如薰，甘露如醴。”
	“雷雨窈冥而未半，皦日笼光於绮寮。”
	“明月松间照，黄河入海流。”（PS：我古诗背错了，应该是清泉石上流）
	“日往菲薇，月来扶疏。”

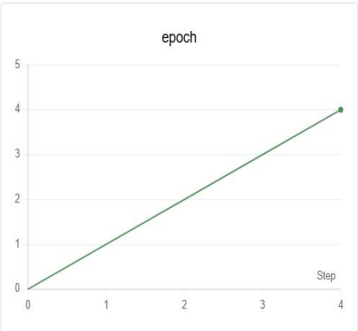


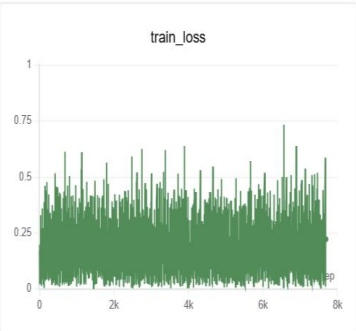
根据 prompt 生成图片的推理过程

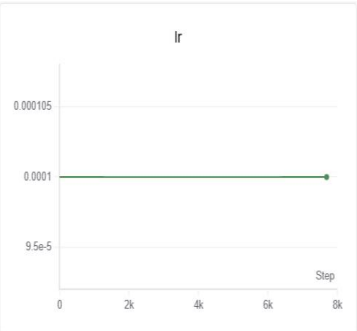
于是又训练了一次。第一次训练输入的预训练模型是实验指导中提供的 SD-v1-5 模型，第二次训练我直接把第一次训练完成的模型加载进去了，相当于第二次是在第一次模型的基础上再训 5 个 epoch，而不是在原模型上训 10 个 epoch，大大减少了实验时间。第二次训练结果就好很多了，很贴近水墨画的风格。训练结果如下：（输入的 prompt 没变，刚好可以和第一次训练结果做对比）

图片	Prompt
	“孤帆远迎碧空尽，唯见长江天际流。”
	“蕙风如薰，甘露如醴。”
	“雷雨窈冥而未半，皦日笼光於绮寮。”
	“明月松间照，黄河入海流。”（PS：我古诗背错了，应该是清泉石上流）
	“日往菲薇，月来扶疏。”

最后在 swanlab 中记录的训练过程图表如下：







然后通过在网上查阅了一些资料，也看了别人训练的模型结果，好像 SD 模型训练过程的特点就是 train-loss 会跳跃变化。

上图是 swanlab 记录的验证集生成的图像。其实设置步数的话可以可视化每张图片的生成过程（但是我训练的时候忘记设置了，所以步数默认是 0，直接展示最终生成的图片）。验证集和我通过 predict.py 中输入相同 prompt 产生的图像不同，这也挺符合日常大模型的使用感受的，即问同样的问题，未必会得到一模一样的答案。

选做内容：“分析模型的实际效果，指出如何改进模型。进一步提升效果”

根据上述两次训练的结果对比，模型的实际效果是越来越好的。与此同时我在网上看到的一些训练的比较好的模型的 batch_size 都在 32 左右，然后 epoch 高达 100，最终的结果就非常震撼。所以改进模型的话就是从训练参数改进起，增加 batch_size 和 epoch（当然 batch_size 和 epoch 也不是越大越好，因为存在过拟合的风险，但这个就需要不断地实验然后得到最优的训练参数组合了），或者也可以在训练过程中动态调整 learning-rate（我两次实验的 lr 都是固定的，没有修改）。

然后由于时间关系，没有再挂别的模型进行训练了（因为其实改代码 bug 和配环境的问题浪费了很多时间）。如果要训练模型生成写实画作，就给它喂写实画作的数据集；如果要换其他模型，那就把其他模型的预训练模型输给训练脚本即可，方法和此次实验的方法并没有什么区别。

上述就是所有的必做实验内容和选做实验内容。最后关于我提交的模型如何复现，我在此作以说明。

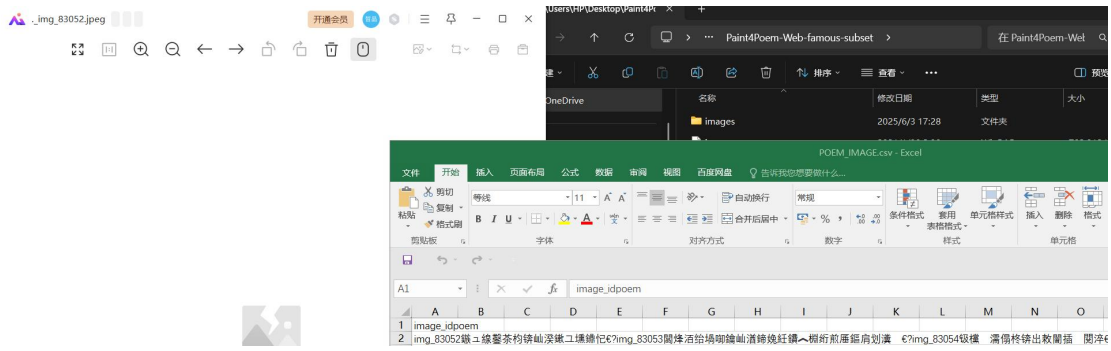
因为整个模型全量微调太大了，没办法交到作业平台。结合我的训练特点（只训练 UNet 部分，VAE 和文本编码器没训练，冻结了），所以只上传了 UNet 的压缩包，只需将实验指导中提供的预训练模型中的 UNet 文件夹替换成我上传的 UNet 文件夹即可运行 predict.py 体验我训练的大模型效果。若要检验模型效果，请将所有文件按照以下方式组织后运行 predict.py：

```
./autodl-tmp/
 |__sd-model-finetuned/          #将实验指导中的 stable-diffusion-v1-5 解压改成这个名字
      |__unet/                   #将 unet 换成我提供的 unet
      |__vae/                    #不变
      |__ text_encoder/          #不变
      |__tokenizer/              #不变
      |__scheduler/              #不变
      |__model_index.json        #不变
./train_sd1-5_Paint4Poem.py       #我提供的训练脚本
./sd_config.py                    #提供的训练参数
./predict.py                      #推理脚本，修改 prompt 运行即可
./requirements.txt                #必要的库
```

5.总结与分析

5.1 问题与解决方案

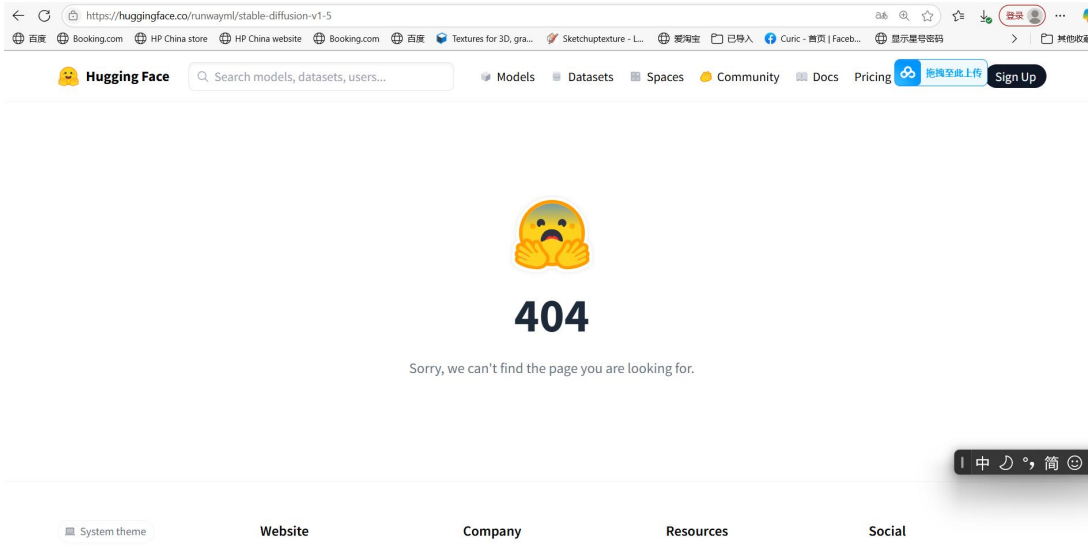
Q1：本来是想使用提供的第一种“图片+标签”格式的数据集，但是下载后的 csv 文件打开乱码，图片显示破损。



S1：csv 打开乱码是因为 excel 本身打开的编码有问题 用 vscode 在 utf-8 下打开就正常了 然后图片文件是因为解压后的有._img_83053.jpeg 这种 MacOS 生成的文件 用图片管理器打不开 往下翻翻就有可以正常打开的原图片了。（但是等我发现这个问题如何解决的时候我已经用 parquet 把模型训练好了。）

Q2：其他就是在运行训练脚本时遇到的一些问题，根据报错进行修改即可。其中特别重要的

一点是，我在网上看到了可以直接从 HuggingFace 官网云端挂载 SD-v1-5 的模型（因为将实验指导中的模型压缩包上传到远程服务器需要 2h，本来想直接从云端挂镜像），但是这个镜像目前已经被官方取消了！



S2：其实有其他的镜像网站，但是使用不了，只能通过上传模型压缩包的方法。

5.2 收获与思考

其实通过这次自行部署并训练大模型的实验，我感觉自己收获到了很多（虽然过程非常磨人）。之前只停留在使用大模型的阶段，只有当自己训练出一个大模型才能真正感受到大模型的魅力。尤其看到模型根据 prompt 生成的水墨画，真的是不一样的感觉。

一开始我首先去了解了 SD 模型，发现 SD 模型的核心部分是 UNet 模型。UNet 模型是我在处理医学图像分割的时候经常使用和见到的模型，竟然也可以用到文生图的任务中。然后就去了解了 UNet 和文生图任务的关联，这是深度学习模型迁移和泛化能力的一个很好例子。

2. 为什么U-Net适合扩散模型/文生图？

扩散模型（如Stable Diffusion）中的核心任务是“去噪”，即：

- 输入一张噪声图片，输出一步“去噪”后的图片。
- 这个过程其实和“分割”很像：输入、输出都是图片，且需要融合全局和局部信息。

U-Net的结构特点恰好满足：

- 编码器提取全局语义，解码器恢复局部细节。
- 跳跃连接让低层细节信息直接传递到高层，有助于“还原”清晰图片，或者保留噪声与原图的某种对应关系。
- 可以方便地接入“条件信息”(如文本编码)，让去噪过程受控。

然后在了解 SD 模型原理的时候，发现 SD 模型的数学基础竟然是马尔科夫链，这在 15 周的课上刚刚讲过，后来就结合课上所讲和相关资料中的解释，更好地理解了 SD 的模型原理。

最后又出于好奇，了解到了 HuggingFace 是一个专注于人工智能（AI）和自然语言处理（NLP）的公司，也是全球最大的 AI 开源社区之一。并且在 HF 的开源社区里发现了很多很有意思的预训练模型和在这些模型上进行的改进。比如，有一个案例是基于 SD-v1-5 模型训练出一个可以生成火影忍者风格的文生图模型。生成的图片也很有意思。

提示词： Lebron James with a hat（戴帽子的勒布朗·詹姆斯）

最后经过此次的实验，我也对训练一个大模型的完整流程有了自己的理解和经验。回到本次实验的算法，实际上没有太多的技术含量，因为主要自己编写的地方就是训练脚本的数据处理和修改参数。

个人觉得本次大模型训练最成功的设计就是，在提供的 SD-v1-5 预训练模型的基础上训练 5 个 epoch 后，又在第一次训练得到的模型上训练 5 个 epoch，没有盲目地在第一次训练效果不好后就直接在原预训练模型重新训练 10 个 epoch，大大减少了训练时间。并且借助 swanlab 全程监控整个训练过程，使得训练过程可视化，并且提供的日志可以查看训练过程中的问题，而且提供了 checkpoint 的功能。这里又要多说一点，checkpoint 在大型训练任务中非常非常重要。因为我是远程连接的服务器，当我的电脑进入待机后都有中断训练的可能（其实可以从提供的 swanlab 网址中看到，第一次训练多次中断，就是因为我的电脑待机或者其他原因导致的训练中断），那么重启训练的时候，如果没有设置 checkpoint 的话，就需要重新训练（这非常恐怖。）。但是设置 checkpoint 就可以从保留的最近断点处继续训练（我在参数中设置的是每 500step 保存一次断点，总共有 7650 步左右，500step 保存一次是比较合理的）。

这次实验不仅让我对大模型有了更深入的理解，也让我认识到了持续学习和适应新技术的重要性。随着人工智能领域的快速发展，我期待将这些经验应用到未来的学习中，解决更复杂的问题。

6.参考文献（资料）

1.训练过程指导：

https://docs.swanlab.cn/examples/stable_diffusion.html

2.扩散模型（Diffusion Model）和稳定扩散模型（Stable Diffusion Model）相关资料：

https://zhuanlan.zhihu.com/p/624221952

https://blog.csdn.net/qq_44648285/article/details/143316237

3.Autodl 使用指导：

AutoDL 使用教程：1）创建实例 2）配置环境+上传数据 3）PyCharm2021.3 专业版下载安装与远程连接完整步骤 4）实时查看 tensorboard 曲线情况-CSDN 博客

4.训练全程图表记录：

https://swanlab.cn/@wxj66666/SD-Paint4Poem/runs/xo2d3thnyu52f9jq2xy71/chart